

EXPERIMENT : 3

C PROGRAMMING : DEBUGGING TOOLS

1. FUNDAMENTALS OF ERROR AND DEBUGGING

Key definitions -

- Error: A mistake or blunder made by the programmer in the source code. Example: Writing $a + b$ when the intention was to multiply $a * b$.
- Bug: A flaw or defect in a computer program that causes it to produce an incorrect or unexpected result, or to behave in unintended ways. Example: The program outputs the sum of two numbers instead of their product.
- Debugging: The systematic process of finding, analyzing, and removing bugs or defects in a computer program.

2. CLASSIFICATION OF ERRORS IN C

- Syntax Errors: Code violates the grammatical rules of the C language. Caused by missing semicolons, unmatched braces, or typos in keywords. Example: `int a = 10` (missing semicolon).
- Compilation / Type Errors: Operations performed on incompatible types or missing declarations. Caused by assigning a float/string to an integer pointer, or using undeclared variables. Example: `int *p = "hello";`
- Linker Errors: The compiler generates object files, but the linker cannot combine them. Caused by declaring a function without defining it, or missing libraries. Example: undefined reference to 'main'
- Runtime Errors: The program crashes or terminates abnormally during execution. Caused by division by zero or accessing invalid memory addresses. Example: `int x = 10 / 0;`
- Logical Errors: Code runs to completion without crashing, but the output is incorrect. Caused by wrong mathematical formulas or incorrect loop conditions. Example: `for(i=0; i<5; i++)` instead of `i<=5`.

Why does a program compile successfully but produce incorrect output?

A compiler's job is to verify that your code obeys the syntax and type rules of the C language, and to convert that code into machine instructions. The compiler cannot understand human intent. If you write `sum = a - b;` instead of `sum = a + b;`, the statement is completely valid C syntax. Therefore, compilation succeeds, but the resulting logic produces an erroneous output at execution time.

Practical Cases in C Debugging

CASE 1 : Syntax & Compilation Errors

When the compiler encounters invalid code, it prints diagnostics specifying the file name, line number, column, and error description.

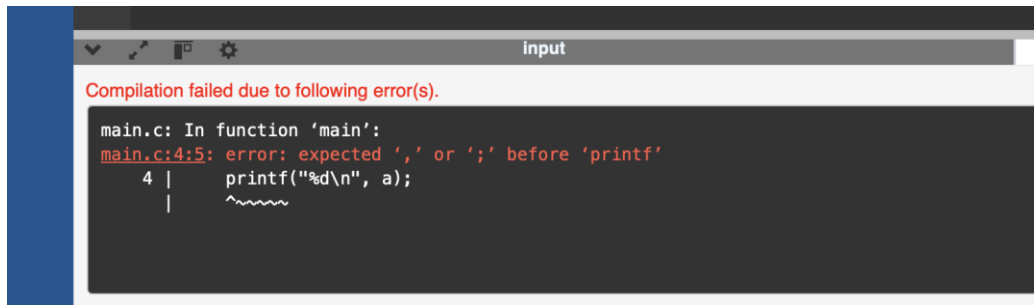
The Problem Code (sample1.c):

```
#include <stdio.h>

int main()
{
    int a = 10    // <-- Missing semicolon here!

    printf("%d\n", a);

    return 0;
}
```



```
input
Compilation failed due to following error(s).
main.c: In function 'main':
main.c:4:5: error: expected ',' or ';' before 'printf'
4 | printf("%d\n", a);
  | ^~~~~
```

Compilation Command & Diagnostic Output: `$ cc sample1.c` sample1.c:5:5: error: expected ';' before 'printf'

- **Decoding the Error Message:**
- **Filename:** sample1.c (the file with the issue)
- **Line & Column:** Line 5, Column 5 (where the compiler got stuck)
- **Type:** error (a fatal issue—code won't build)
- **Description:** expected ';' or ';' before 'printf' (it's reminding you that you forgot a semicolon before the printf line)

Case 2: Compiler Warnings (-Wall, -Wextra)

A warning indicates code that is syntactically legal, but potentially dangerous or suspicious. Warnings do not stop compilation, but ignoring them frequently leads to bugs at runtime.

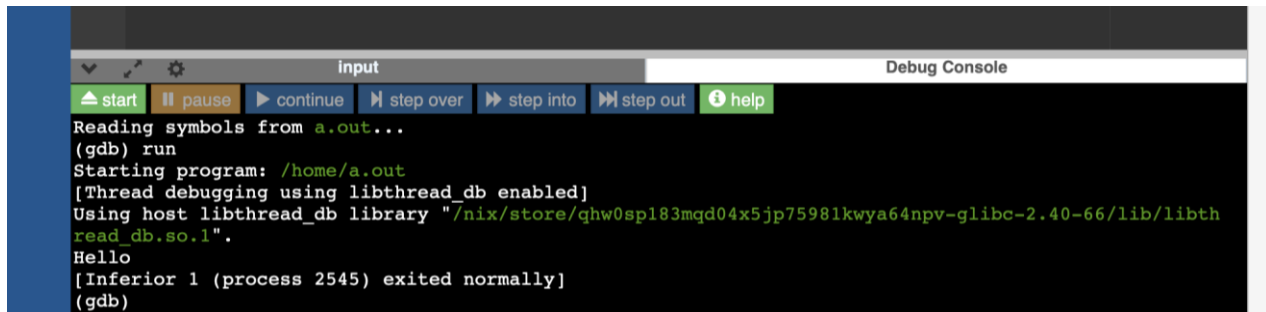
The Code (sample2.c):

```
#include <stdio.h>

int main()
{
    int a; // <-- Variable 'a' is declared but never used!

    printf("Hello\n");

    return 0;
}
```



```
input Debug Console
start pause continue step over step into step out help
Reading symbols from a.out...
(gdb) run
Starting program: /home/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/nix/store/qhw0sp183mqd04x5jp75981kwya64npv-glibc-2.40-66/lib/libth
read_db.so.1".
Hello
[Inferior 1 (process 2545) exited normally]
(gdb)
```

- **Standard Compilation:** Running `cc sample2.c` suppresses minor warnings, compiles without complaint, and lets you run `./a.out`.
- **Enabling Warning Flags:** Running `cc -Wall -Wextra sample2.c` forces the compiler to give you full visibility:
 - **-Wall:** Turns on standard, highly useful warnings.
 - **-Wextra:** Turns on additional strict checks not caught by `-Wall`.
 - *Example Output:* `sample2.c:5:9: warning: unused variable 'a' tells you a was declared but never used.`

Case 3: printf-Based Debugging (Logical Bugs)

Before using complex tools, drop `printf()` statements into your code to watch variables update in real-time.

Code (sample3.c):

```
#include <stdio.h>

int main(void) {

    int i, sum = 0;

    for(i = 0; i < 5; i++) { // Logical bug: stops before 5

        printf("DEBUG: entering iteration i = %d\n", i);

        sum = sum + i;
```

```

    printf("DEBUG: sum updated to = %d\n", sum);
}

printf("Final Sum = %d\n", sum);

return 0;

}

```

```

input
Debug Console
start pause continue step over step into step out help
DEBUG: entering iteration i = 2
DEBUG: sum updated to = 3
DEBUG: entering iteration i = 3
DEBUG: sum updated to = 6
DEBUG: entering iteration i = 4
DEBUG: sum updated to = 10
Final Sum = 10
[Inferior 1 (process 2106) exited normally]
(gdb)

```

- **Why Print Statements Help:** Printing *i* and *sum* step-by-step exposes that the loop stops at *i* = 4 instead of going up to 5.

Case 4: Debugging a Logical Error using GDB

instead of manually adding `printf()` lines, use GDB to watch execution.

Code (sample4.c):

```

#include <stdio.h>

int main() {

    int i, sum = 0;

    for(i = 1; i < 5; i++) { // Bug: condition should be i <= 5

        sum = sum + i;

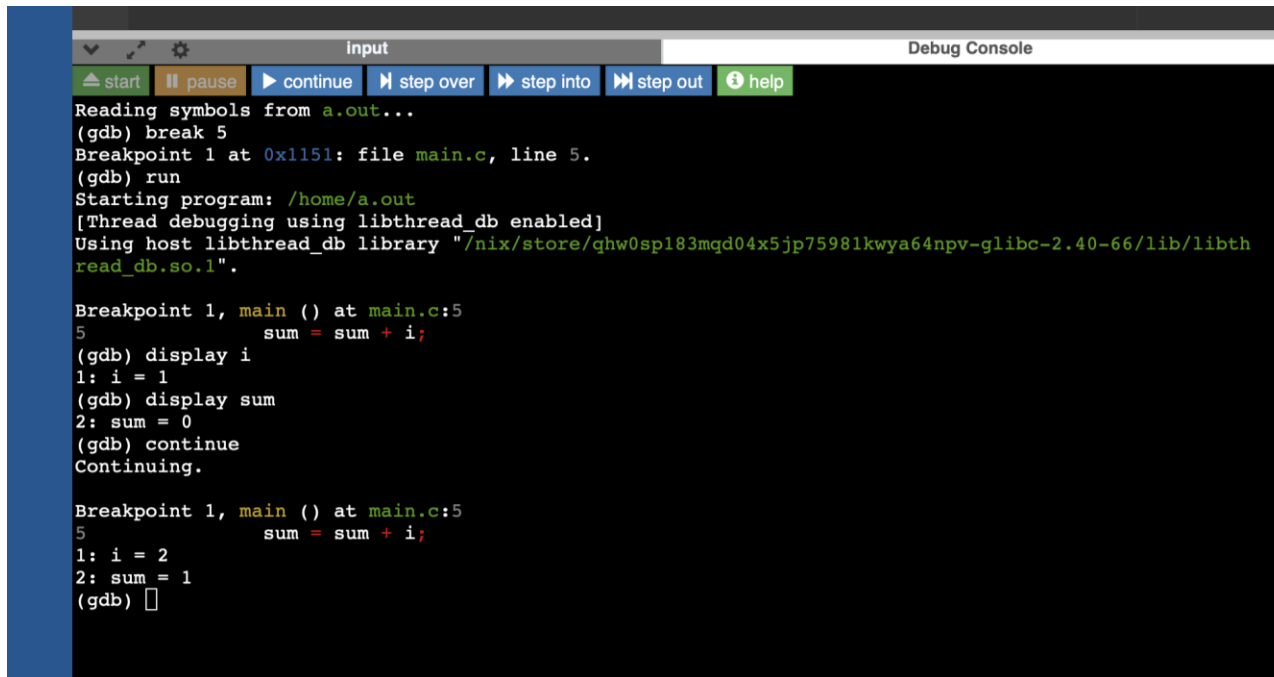
    }

    printf("Sum = %d\n", sum);
}

```

```
return 0;

}
```



```
input Debug Console
start pause continue step over step into step out help
Reading symbols from a.out...
(gdb) break 5
Breakpoint 1 at 0x1151: file main.c, line 5.
(gdb) run
Starting program: /home/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/nix/store/ghw0sp183mqd04x5jp75981kwy64npv-glibc-2.40-66/lib/libth...read_db.so.1".

Breakpoint 1, main () at main.c:5
5      sum = sum + i;
(gdb) display i
1: i = 1
(gdb) display sum
2: sum = 0
(gdb) continue
Continuing.

Breakpoint 1, main () at main.c:5
5      sum = sum + i;
1: i = 2
2: sum = 1
(gdb) 
```

- **Expected vs Actual:** Expected 15 (1+2+3+4+5), but got 10 (1+2+3+4).
- **GDB Walkthrough:**
 3. Compile with debug symbols: `cc -g sample4.c -o sample4`
 4. Launch GDB: `gdb ./sample4`
 5. Set a breakpoint: `(gdb) break 7`
 6. Run and set watchers: `(gdb) run` then `(gdb) display i` and `(gdb) display sum`
 7. Use `continue` repeatedly. You'll observe the loop exits when `i = 5` without adding 5 to `sum`.
 8. **Fix:** Change `i < 5` to `i <= 5`.

Case 5: Debugging Functions (next vs. step)

When your code hits a function call, you have two choices in GDB:

- **next (Step Over):** Runs the function in the background and moves to the next line in main().
- **step (Step Into):** Jump inside the function body line-by-line.

- **Code (sample5.c):**

```
#include <stdio.h>
```

```
int square(int x) {
```

```
    int result1;
```

```
    result1 = x * x;
```

```
    return result1;
```

```
}
```

```
int main() {
```

```
    int result, n = 5;
```

```
    result = square(n);
```

```
    printf("Square = %d\n", result);
```

```
    return 0;
```

```
}
```



```
input Debug Console
start pause continue step over step into step out help
Reading symbols from a.out...
(gdb) break main
Note: breakpoint 1 also set at pc 0x1156.
Breakpoint 3 at 0x1156: file main.c, line 8.
(gdb) run
Starting program: /home/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/nix/store/qhw0sp183mqd04x5jp75981kwy64npv-glibc-2.40-66/lib/libth...read_db.so.1".

Breakpoint 1, main () at main.c:8
8      int result, n = 5;
(gdb) next
9      result = square(n);
(gdb) step
square (x=5) at main.c:4
4      result1 = x * x;
(gdb)
```

- **Option A (next):** Stepping over square(n) directly returns result = 25 without entering square().
- **Option B (step):** Stepping into square(n) takes GDB straight into square(x=5) at line 5 so you can debug inside square().

Case 6: Debugging Runtime Errors using Backtrace (backtrace / bt)

When a program crashes (e.g., floating point exception / division by zero), GDB can pinpoint the exact function call sequence leading to the crash.

Example Code (sample6.c):

```
#include <stdio.h>

int divide(int a, int b) {
    return a / b;    // Division by zero when b = 0
}

int calculate(int x) {
    return divide(x, 0);    // Passes 0 as divisor
}

int main() {
    int result = calculate(10);
```



```
    printf("Result: %d\n", result);  
    return 0;  
}
```

GDB Backtrace Execution:

```
$ cc -g sample6.c -o sample6
```

```
$ gdb ./sample6
```

```
(gdb) run
```

Program received signal SIGFPE, Arithmetic exception.

```
0x000055555555132 in divide (a=10, b=0) at sample6.c:3
```

```
3    return a / b;
```

```
(gdb) backtrace
```

```
#0 0x000055555555132 in divide (a=10, b=0) at sample6.c:3
```

```
#1 0x000055555555151 in calculate (x=10) at sample6.c:7
```

```
#2 0x00005555555516a in main () at sample6.c:11
```

The screenshot shows a GDB Debug Console window with a dark background. At the top, there's a toolbar with buttons for start, pause, continue, step over, step into, step out, and help. Below the toolbar, the console displays the following text:

```
Reading symbols from a.out...
(gdb) info local
No frame selected.
(gdb) info break
Num      Type      Disp Enb Address      What
1        breakpoint keep y  0x00000000000001170 in main at main.c:9
2        breakpoint keep y  0x00000000000001157 in calculate at main.c:6
(gdb) run
Starting program: /home/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/nix/store/qhw0sp183mqd04x5jp75981kwya64npv-glibc-2.40-66/lib/libth
read_db.so.1".
Breakpoint 1, main () at main.c:9
9      int result = calculate(10);
(gdb) backtrace
#0  main () at main.c:9
(gdb)
```

Case 7: Debugging Array Out-of-Bounds Errors using GDB

In C, array bounds are not checked at runtime automatically. Accessing indices beyond the declared limit results in undefined behavior or memory corruption.

Example Code (sample7.c):

```
#include <stdio.h>
```

```
int main() {
```

```
    int a[5];
```

```
    int i;
```

```
    for(i = 0; i <= 5; i++) {
```

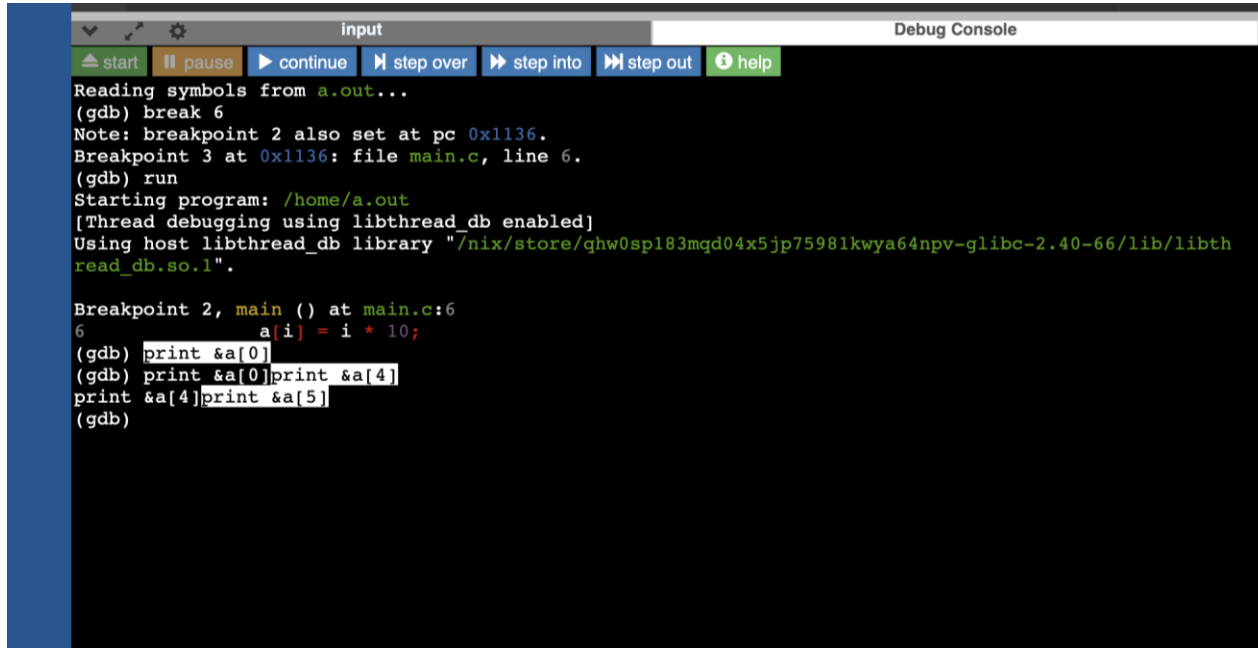
```
        scanf("%d", &a[i]); // Line 7 - Loop runs 6 times for size 5!
```

```
    }
```

```
    return 0;
```

```
}
```

inspection in GDB:



The screenshot shows a GDB debug console window. At the top, there is a toolbar with buttons for start, pause, continue, step over, step into, step out, and help. Below the toolbar, the console displays the following text:

```
Reading symbols from a.out...
(gdb) break 6
Note: breakpoint 2 also set at pc 0x1136.
Breakpoint 3 at 0x1136: file main.c, line 6.
(gdb) run
Starting program: /home/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/nix/store/qhw0sp183mqd04x5jp75981kwy64npv-glibc-2.40-66/lib/libth
read_db.so.1".

Breakpoint 2, main () at main.c:6
6      a[i] = i * 10;
(gdb) print &a[0]
(gdb) print &a[0]print &a[4]
print &a[4]print &a[5]
(gdb)
```

Case 8: Segmentation Faults (SIGSEGV) using GDB

A Segmentation Fault occurs when a program attempts to access a memory location that it does not have permission to access (such as dereferencing a NULL pointer).

Example Code (sample8.c):

```
#include <stdio.h>
```

```
int main() {
```

```
    int *p = NULL;    // Pointer points to address 0x0
```

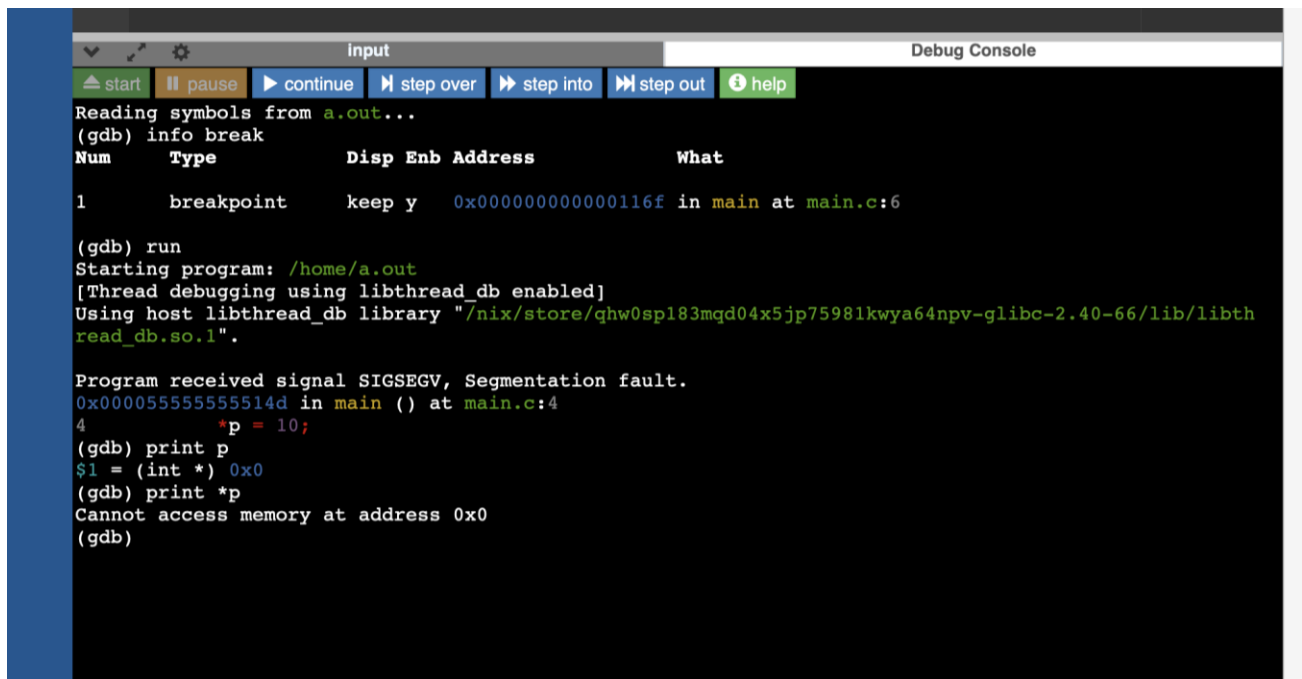
```
    *p = 10;          // Attempting to write to address 0x0 causes Segmentation Fault
```

```
    printf("%d\n", *p);
```

```
    return 0;
```

```
}
```

Debugging in GDB:



The screenshot shows a GDB debug console window. At the top, there is a toolbar with buttons for start, pause, continue, step over, step into, step out, and help. Below the toolbar, the text "Reading symbols from a.out..." is displayed. The user has entered the command "(gdb) info break", and the console shows a table of breakpoints:

Num	Type	Disp	Enb	Address	What
1	breakpoint	keep	y	0x000000000000116f	in main at main.c:6

After entering "(gdb) run", the program starts. The console shows the following output:

```
Starting program: /home/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/nix/store/qhw0sp183mqd04x5jp75981kwy64npv-glibc-2.40-66/lib/libth
read_db.so.1".

Program received signal SIGSEGV, Segmentation fault.
0x000055555555514d in main () at main.c:4
4      *p = 10;
(gdb) print p
$1 = (int *) 0x0
(gdb) print *p
Cannot access memory at address 0x0
(gdb)
```

Corrected Code:

```
#include <stdio.h>
```

```
int main() {
```

```
    int x;
```

```
    int *p = &x; // Assign p to point to valid memory address of x
```

```
    *p = 10; // Valid write
```

```
    printf("%d\n", *p);
```

```
    return 0;
```

```
}
```

- *Systematic Debugging Strategy*

To debug effectively, follow this 8-step methodology:

- 1. Observe Expected vs. Actual Output*
- 2. Reproduce the Issue Consistently*
- 3. Compile with Flags (-Wall -Wextra -g)*
- 4. Isolate Suspicious Code Sections*
- 5. Inspect State (printf or GDB breakpoints)*
- 6. Apply a Single Minimal Fix*
- 7. Recompile and Re-test*
- 8. Verify with Edge Cases & Boundary Inputs*

- **EXERCISE AND HOMEWORK QUESTIONS**

1. ERROR CLASSIFICATION :

Syntax Error (Missing semicolon):

```
#include <stdio.h>
int main() {
    int x = 10
    printf("%d\n", x);
    return 0;
}
```

Compilation / Type Error (Type mismatch):

```
#include <stdio.h>

int main() {

    int *ptr = "Hello World";

    return 0;

}
```

Linker Error (Missing function definition):

```
#include <stdio.h>
```

```
void missingFunction(); // Declared but not defined
```

```
int main() {  
    missingFunction();  
    return 0;  
}
```

Runtime Error (Division by zero):

```
#include <stdio.h>
```

```
int main() {  
    int a = 10, b = 0;  
    int result = a / b;  
    return 0;  
}
```

Logical Error (Incorrect loop condition):

```
#include <stdio.h>
```

```
int main() {  
    // Intended to loop 5 times, but loops 4 times  
    for (int i = 0; i < 4; i++) {  
        printf("Iteration %d\n", i);  
    }  
    return 0;  
}
```

2. GNU Concept

- **GNU Project:** Initiated by Richard Stallman in 1983, the GNU Project aims to give computer users freedom and control by developing a completely free, open-source Unix-like operating system.
- **Role of GDB:** GDB (GNU Debugger) is the standard source-level debugger for the GNU software ecosystem. It allows developers to inspect program execution, view variables, and analyze memory during runtime or post-crash analysis.

3. Shell Context

- **Kernel:** The central core component of an operating system that manages hardware resources (CPU, memory, storage) and mediates communication between software applications and system hardware.
- **Shell:** A user-interface program (command-line or graphical) that acts as an interpreter, taking input commands from the user and translating them into instructions for the kernel to execute.
- **Three Common Linux Shells:**
 - Bash (Bourne Again Shell)
 - Zsh (Z Shell)
 - Fish (Friendly Interactive Shell)

4. Differences: Breakpoint, Watchpoint, Catchpoint, and Tracepoint

- **Breakpoint:** Pauses execution when the program reaches a specific source line, function, or memory address.
- **Watchpoint:** Pauses execution whenever the value of a specified variable or memory location changes (or is read).
- **Catchpoint:** Pauses execution when a specific C++ exception or system event occurs (e.g., throwing an exception, catching a signal, or loading a dynamic library).
- **Tracepoint:** Logs specific program data at specified points without pausing or stopping program execution (commonly used in remote or performance-critical debugging).

